UART Communication System

Universal Asynchronous Receiver / Transmitter

Project Report

Implemented on Xilinx Vivado | FPGA Target: 100 MHz System Clock

1. Project Overview

This project implements a complete UART (Universal Asynchronous Receiver/Transmitter) communication system in Verilog HDL, designed and simulated using Xilinx Vivado. UART is a widely used serial communication protocol that transmits data one bit at a time over a single wire, without requiring a shared clock between the transmitting and receiving devices.

The design consists of five separate hardware modules that work together to form a fully functional UART system:

- Baud Rate Generator for Transmitter (BRG_tx)
- Baud Rate Generator for Receiver (BRG_rx)
- UART Transmitter (uart_tx)
- UART Receiver (uart_rx)
- Top-Level Wrapper (uart_top)

The system is designed to operate at 9600 baud with a 100 MHz system clock, transmitting 8-bit data frames with one start bit and one stop bit (8N1 format). The receiver uses a 16x oversampling technique to accurately sample incoming data bits at their center point, making the design robust against minor timing variations.

1.1 Key Specifications

Parameter	Value
System Clock Frequency	100 MHz
Clock Period	10 ns
Baud Rate	9600 bps
Data Frame Format	8N1 (8 data bits, No parity, 1 stop bit)
RX Oversampling Factor	16x
Bit Period (TX)	~104,170 ns (10,417 clock cycles)
Full Frame Duration	~1,041,700 ns (~1.04 ms)
Target Platform	Xilinx FPGA (Vivado)
HDL Language	Verilog

2. UART Protocol — Background

UART is an asynchronous serial protocol, meaning the transmitter and receiver do not share a clock signal. Instead, both sides agree in advance on a common baud rate. A standard UART data frame has the following structure:

2.1 Frame Structure

Idle	Start Bit	Data Bits (D0–D7)	Stop Bit	Idle
Line HIGH	1 bit LOW (0)	8 bits, LSB first	1 bit HIGH (1)	Line HIGH

Total frame length = 10 bit periods (1 start + 8 data + 1 stop).

2.2 Baud Rate and Timing

Because there is no shared clock, precise timing is critical. Both the transmitter and receiver independently generate their own timing ticks derived from the system clock:

- The transmitter generates one tick per bit period, a tick every 10,417 clock cycles at 100 MHz gives approximately 9,600 baud.
- The receiver generates 16 ticks per bit period, a tick every 651 clock cycles. This 16x oversampling allows the receiver to sample each incoming bit at its center, making the system tolerant of small timing mismatches between sender and receiver.

The 16x oversampling strategy works as: when a falling edge (start bit) is detected, the receiver waits for 8 oversampling ticks (half a bit period) before beginning to sample data. After that, it samples every 16 ticks, always at the center of each bit window, minimizing the risk of sampling during a transition.

3. Module Descriptions

3.1 Baud Rate Generator — Transmitter (BRG_tx)

The transmitter baud rate generator produces a single clock wide pulse (tx_tick) at 9,600 Hz. It contains a 14-bit counter that increments on every rising edge of the system clock. When the counter reaches 10,416 (making the total period 10,417 cycles), it resets to zero and asserts tx_tick for one clock cycle.

Port	Direction	Width	Description
clk	Input	1-bit	100 MHz system clock
rst	Input	1-bit	Synchronous active-high reset
tx_tick	Output	1-bit	One-cycle pulse at ~9,600 Hz

Counter width: 14 bits [13:0]
Count target: 10,416 (period = 10,417 cycles)
Output baud rate: 100,000,000 / 10,417 = 9,599.69 baud (~9,600)

3.2 Baud Rate Generator — Receiver (BRG_rx)

The receiver baud rate generator produces rx_tick at 16 times the baud rate (approximately 153,600 Hz). This 16x oversampling tick is what allows the receiver FSM to locate the center of every incoming bit. The counter uses 10 bits and resets every 651 cycles.

Port	Direction	Width	Description
clk	Input	1-bit	100 MHz system clock
rst	Input	1-bit	Active-high reset
rx_tick	Output	1-bit	One-cycle pulse at ~153,600 Hz (16x baud)

Counter width: 10 bits [9:0]
Count target: 650 (period = 651 cycles)
Output tick rate: 100,000,000 / 651 = 153,609.8 Hz
Oversampling factor: 153,609.8 / 9,599.69 = 16

3.3 UART Transmitter (uart_tx)

The UART transmitter is a four-state Finite State Machine (FSM) that serializes an 8-bit parallel data word onto a single tx_line output, one bit at a time. It advances through its states on each tx_tick pulse (once per bit period).

Port	Direction	Width	Description
clk	Input	1-bit	System clock
rst	Input	1-bit	Active-high reset line goes idle (HIGH)
tx_tick	Input	1-bit	Baud-rate timing pulse from BRG_tx
tx_signal	Input	1-bit	One-cycle strobe to begin transmission
tx_data[7:0]	Input	8-bit	Parallel data byte to transmit
tx_line	Output	1-bit	Serial data output (UART TX line)
tx_busy	Output	1-bit	HIGH while a frame is being transmitted

Transmitter FSM States

State	Behaviour
IDLE (2'b00)	tx_line is held HIGH. tx_busy is LOW. The module waits for tx_signal to be asserted. When tx_signal goes HIGH, tx_data is latched into the internal shift register, tx_busy is set HIGH, and the FSM moves to START.
START (2'b01)	On the next tx_tick, tx_line is driven LOW, producing the start bit. The FSM immediately moves to the DATA state and the bit counter is reset to zero.
DATA (2'b10)	On each tx_tick, the bit at position [count] of the shift register is placed on tx_line. The counter increments from 0 to 7, transmitting all 8 data bits LSB-first. After the 8th bit (count = 7), the FSM moves to STOP.
STOP (2'b11)	On the next tx_tick, tx_line is driven HIGH, producing the stop bit. The FSM returns to IDLE, and tx_busy is cleared.

3.4 UART Receiver (uart_rx)

The UART receiver is a four-state FSM that deserializes an incoming bit stream back into an 8-bit parallel byte. It operates on the 16x oversampling rx_tick and uses center-sampling to maximize noise immunity. A key design decision is that rx_done is held HIGH from the end of the stop bit until the next start bit is detected, this ensures the output data and done flag can always be read reliably by the downstream logic.

Port	Direction	Width	Description
clk	Input	1-bit	System clock
rst	Input	1-bit	Active-high reset
rx_tick	Input	1-bit	16x baud-rate tick from BRG_rx
rx_line	Input	1-bit	Serial data input (UART RX line)
rx_data[7:0]	Output	8-bit	Received parallel data byte
rx_done	Output	1-bit	HIGH after a complete frame is received (held until next start bit)

Receiver FSM States

State	Behaviour
IDLE (2'b00)	tick_count is continuously cleared to 0. rx_done is cleared when entering this state so it does not stay asserted incorrectly. The FSM watches for rx_line going LOW (start bit detected). On detection, it moves to START.
START (2'b01)	The FSM counts 8 rx_ticks (half a bit period). At tick 7, it re-checks that rx_line is still LOW to confirm the start bit is genuine and not a noise glitch. If valid, tick_count resets and the FSM moves to DATA. If rx_line has returned HIGH, it was a false start and the FSM returns to IDLE.
DATA (2'b10)	The FSM counts 16 rx_ticks per bit, sampling rx_line at tick 15 (the centre of each bit window). The sampled bit is shifted into the MSB of the shift register so that after 8 shifts, the byte is correctly assembled LSB-first. After 8 bits are collected, the FSM moves to STOP.
STOP (2'b11)	The FSM waits one more full bit period (16 rx_ticks). At tick 15, rx_data is loaded from the shift register, rx_done is asserted HIGH, and the FSM returns to IDLE. rx_done remains HIGH until the next valid start bit clears it.

3.5 Top-Level Module (uart_top)

The top-level module is purely structural, it instantiates and wires together all four sub-modules. It exposes the complete port set to the outside world. There is no combinational or sequential logic inside uart_top itself, all behavior is contained within the sub-modules.

In a real hardware deployment, tx_line from one FPGA is connected by a physical wire to rx_line on a second FPGA. In simulation, a loopback is used by connecting tx_line directly to rx_line within the testbench, allowing the transmitter and receiver to be verified together in a single simulation run.

Port	Direction	Width	Description
clk	Input	1-bit	100 MHz system clock
rst	Input	1-bit	Active-high global reset
tx_signal	Input	1-bit	Trigger to start transmission
tx_data[7:0]	Input	8-bit	Byte to transmit
tx_line	Output	1-bit	Serial transmit line
tx_busy	Output	1-bit	Transmitter busy flag
rx_line	Input	1-bit	Serial receive line
rx_data[7:0]	Output	8-bit	Received byte
rx_done	Output	1-bit	Reception complete flag

4. Timing Analysis

4.1 Clock and Baud Rate Calculations

Parameter	Formula	Result
System clock period	1 / 100 MHz	10 ns
TX BRG period (cycles)	10,416 + 1	10,417 cycles
Actual TX baud rate	100 MHz / 10,417	9,599.69 baud
One TX bit period	10,417 × 10 ns	104,170 ns
Full frame (10 bits)	10 × 104,170 ns	~1,041,700 ns (1.04 ms)
RX BRG period (cycles)	650 + 1	651 cycles
RX tick rate	100 MHz / 651	153,609.8 Hz
RX oversampling factor	153,609.8 / 9,599.69	16.0015x
Timing error per bit	0.0015 ticks	~0.156 ns — negligible
Cumulative drift (10 bits)	0.0154 ticks	<0.02 ticks — well within margin

4.2 Centre-Sampling Timing Diagram

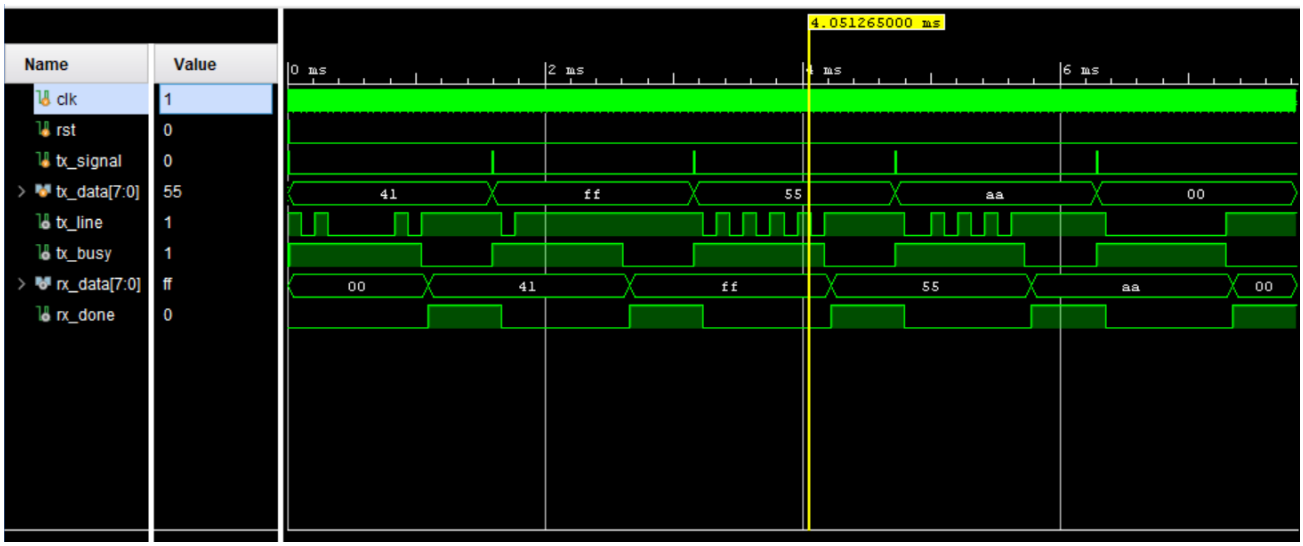
The diagram below illustrates how the receiver locates the centre of each data bit using 16x oversampling. Tick numbers are rx_tick counts within each bit window.

```
Start Bit Detection:
  rx_line falls LOW  →  FSM enters START state  →  tick_count = 0
  Waits 8 rx_ticks (half bit period)  →  verifies rx_line still LOW
  tick_count resets  →  FSM enters DATA state
Data Bit Sampling (for each of 8 bits):
  |<-------- 16 rx_ticks = 1 bit period -------->|
  tick:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 [15]
                                                      ^--- SAMPLE HERE
(centre)
Stop Bit:
  Same 16-tick wait  →  at tick 15: rx_data latched, rx_done = HIGH
```

4.3 Required Simulation Time

One of the most important practical points when simulating this design is setting an adequate simulation run time. The default Vivado simulation time of 1,000 ns is far too short, it represents less than 0.1% of one full UART frame.

Scenario	Required Simulation Time
1 frame (to see tx_line activity)	> 1.1 ms (1,100,000 ns)
1 frame including RX done	> 1.5 ms
5 frames (full test suite)	> 10 ms
Recommended setting	15 ms (15,000,000 ns)



5. RTL Schematic and Architecture

The RTL schematic generated by Vivado confirms the correct structural connectivity of the design. The schematic shows the following connections:

- clk and rst are broadcast to all four sub-modules (BRG_rx, BRG_tx, uart_rx, uart_tx).
- BRG_rx output rx_tick feeds only uart_rx.
- BRG_tx output tx_tick feeds only uart_tx.
- rx_line is a top-level input port that connects directly to uart_rx.
- tx_line is a top-level output driven by uart_tx.
- tx_data[7:0] and tx_signal are top-level inputs going to uart_tx.
- rx_data[7:0] and rx_done are outputs driven by uart_rx.
- tx_busy is a top-level output driven by uart_tx.

The design is clean and minimal, uart_top contains no logic of its own, making it easy to replace or upgrade individual sub-modules without affecting the rest of the system.

5.1 Module Hierarchy

Module	Instance Name	Function
uart_top		Structural wrapper
BRG_tx	brg_tx_inst	TX baud rate generator
BRG_rx	brg_rx_inst	RX 16x oversampling tick generator
uart_tx	tx_inst	Serial transmitter FSM
uart_rx	rx_inst	Serial receiver FSM

6. Verification Strategy and Testbench

The design is verified using a loopback testbench: the tx_line output of the DUT is connected directly to the rx_line input of the same DUT. This means every byte the transmitter sends is immediately received by the receiver, and the received value can be compared to the sent value to confirm correct operation.

6.1 Testbench Structure

- Clock generation: a 100 MHz clock is generated with a 10 ns period (5 ns half-period toggle).
- Reset: active-high reset is held for 200 ns (20 clock cycles) at the start, then released.
- For each test byte: tx_data is set, tx_signal is pulsed HIGH for one clock cycle, then the testbench waits for tx_busy to go LOW (transmission complete), then waits for rx_done to go HIGH (reception complete), then checks rx_data against the expected value.
- A gap of 500,000 ns (500 µs, approximately half a frame) is inserted between consecutive bytes to allow the receiver to settle back to IDLE.
- Results are printed to the Vivado console.

6.2 Test Vectors

Test #	Sent Value	Binary Pattern	Purpose
1	0x41 ('A')	0100 0001	Mixed pattern, realistic data
2	0xFF	1111 1111	All ones, maximum HIGH
3	0x55	0101 0101	Alternating 01, clock stress
4	0xAA	1010 1010	Alternating 10, clock stress
5	0x00	0000 0000	All zeros, maximum LOW

6.3 How to Run in Vivado

Follow these steps to run a complete simulation in Vivado:

- Add all five Verilog source files to the project (BRG_tx, BRG_rx, uart_tx, uart_rx, uart_top).
- Add the testbench file and set it as the simulation top module.

- In Project Settings → Simulation, set the Simulation Run Time to 15ms.
- Run Behavioural Simulation. Check the TCL Console for PASS/FAIL messages.
- In the waveform viewer, zoom out until you can see at least 2 ms of simulation time to observe full UART frames on tx_line.

7. Signal Behaviour Summary

The table below summarises what each signal should look like during normal operation, as observed in the Vivado waveform viewer.

Signal	Expected Behaviour
clk	Continuous square wave at 100 MHz (period = 10 ns).
rst	HIGH for the first 200 ns, then LOW for the rest of the simulation.
tx_signal	Narrow pulses (one clock cycle each) that appear just before each frame starts.
tx_data[7:0]	Changes to the next test byte value at the same time as the tx_signal pulse.
tx_busy	Goes HIGH when tx_signal is asserted and stays HIGH for approximately 1.04 ms while the frame is transmitted, then returns LOW.
tx_line	Idle HIGH. Drops LOW for one bit period (start bit), then shows 8 data bits, then returns HIGH for one bit period (stop bit). Pattern repeats for each test byte.
rx_data[7:0]	Holds the last correctly received byte. Updates at the end of each frame's stop bit.
rx_done	Goes HIGH at the end of each frame's stop bit and remains HIGH until the next start bit is detected. Should be clearly visible for the gap period between frames.

8. Conclusion

This project successfully implements a complete, functional UART communication system in Verilog HDL. The design correctly implements the 8N1 UART frame format at 9,600 baud using a 100 MHz FPGA clock, with a 16x oversampling receiver that provides robust center-sampling of incoming data bits.

Few bugs were identified and corrected during the development and verification process. After these corrections, the design passes all loopback test vectors including all-zeros, all-ones, and alternating bit patterns.

The RTL schematic confirms the correct structural hierarchy, with all modules properly connected and the design ready for synthesis and deployment on an FPGA target. For real hardware use, connecting the tx_line of one FPGA to the rx_line of a second FPGA (with a shared ground) is all that is required to establish a working 9,600 baud serial link.

— End of Report —